RESEARCH MEMORANDUM

Django vs. Flask for the Sleep Study Recruitment Website

Isha Hebbar

2026-10-26

Table of contents

1 Introduction 1

2 Key Analysis 2
- 2.1 SWOT: Django . . . 2
- 2.2 SWOT: Flask . . . 2
- 2.3 Other Options . . . 3
- 2.4 Summary Comparison . . . 3

3 Suggestions for Next Steps 4

References 4

1 Introduction

Researcher: Isha Hebbar
Question: Should the sleep study recruitment website be built with Django or Flask?

Right now, interested students email the lab and a grad student manually sends back eligibility questions. That takes hours each week and people drop out along the way. The site needs to describe the study, run an eligibility screener of about eight questions, collect a name and email from people who qualify, and let research assistants (RAs) log in to see the list of sign-ups.

The constraints that matter most: the grad student knows Python for data analysis but has never built a website, the site stores contact info so security and privacy matter, it should be running in about a month, and nobody will maintain it full time.

Recommendation: Django. It has login, an admin page, form validation, and security protections built in, which covers most of the requirements. This memo compares Django and Flask with a SWOT analysis for each, and also considers FastAPI and a no-code option.

2 Key Analysis

2.1 SWOT: Django

Strengths

- Login and admin are built in, so the RA sign-up list is basically Django's free admin page.
- Security defaults are on: protection against CSRF, SQL injection, and XSS, and hashed passwords (Django Software Foundation 2025).
- One standard way to do things, and built-in forms for the screener.

Weaknesses

- Steeper learning curve for someone who has only done data analysis.
- Heavy for a small site, with many files for about four pages.

Opportunities

- Easy to reuse for later studies; a new study could be a new model in the same project.
- The admin page can filter sign-ups, so RAs may not need a custom dashboard.

Threats

- An unmaintained site can fall behind on security patches.
- Campus IT may need to support a full Django deployment, which could delay the one-month goal.
- If the grad student leaves, the next person has to learn Django too.

2.2 SWOT: Flask

Strengths

- Easy to start: a working page is about five lines of Python.
- Small and readable, and the screener logic can be plain Python.

Weaknesses

- Security is your job. Flask has no built-in login or CSRF protection, so you add extensions like Flask-Login and Flask-WTF (Pallets 2025).
- Many small choices (which database library? which login extension?) and the RA sign-up list has to be built by hand.

Opportunities

- A quick prototype could be shown to the PI within the first week or two.

Threats

- A missed or wrong add-on setup could leak contact info, which campus IT will care about.
- Add-ons are maintained by different people and can go stale, and with no standard layout a future maintainer may not understand the site.

2.3 Other Options

FastAPI. I have used it before and it is easy to write (Ramirez 2025), but it is built for APIs, not pages with forms. It has no built-in login, sessions, or admin page, and it carries the same security risk as Flask. It would make sense if we later want a separate API.

No-code (form + simple page). A university-approved form tool (Microsoft Forms, Qualtrics, or REDCap) could run the screener and sign-up, with a simple UW-hosted page describing the study. This is live in days and IT may have already approved where the data is stored, but the screener logic and design are limited. Commercial site builders like Webflow store data on a third-party server IT has not reviewed. This may be the fastest and safest option, so it is worth raising with IT.

2.4 Summary Comparison

Table 1: Summary comparison for the sleep study site

Question	Django	Flask	FastAPI
Learning curve for a beginner	Steeper	Gentle	Gentle
RA login + sign-up list	Mostly built in	Build it yourself	Build it yourself
Security by default	Strong	Add-ons needed	Add-ons needed
Fits a small one-time site	OK	Great	OK
Easy to reuse and hand off	Great	Depends	Depends

A Django model for a sign-up is only a few lines, which shows why the admin page comes cheap.[1]

```python
from django.db import models

class SignUp(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    created = models.DateTimeField(auto_now_add=True)
```

3 Suggestions for Next Steps

The biggest risks for this project are not build speed. They are security of contact information and handing it off to people who won't maintain it full time, and Django addresses both best. The learning curve is its real cost, so the grad student should start with the official Django tutorial and use the admin page for the RA view instead of building a custom one. If IT can only host something very simple, Flask is the fallback.

Questions to confirm with IT:

1. Would a university-approved form plus a simple page be acceptable instead of a custom site?
2. Can the university host a Python app with a database, and where would it live?
3. Does IT require a specific database (SQLite, PostgreSQL) or encryption at rest?
4. Who will apply security updates after the study starts?
5. Does the IRB need to approve storing screener answers (sleep and health info) for qualified participants?
6. Can someone with web experience help in the first week?

References

Django Software Foundation. 2025. *Django Documentation: Security in Django*. https://docs.djangoproject.com/en/stable/topics/security/.

Pallets. 2025. *Flask Documentation: Security Considerations*. https://flask.palletsprojects.com/en/stable/web-security/.

Ramirez, Sebastian. 2025. *FastAPI Documentation*. https://fastapi.tiangolo.com/.

[1] This is a sketch, not tested code. It would need to be registered in `admin.py` to show up in the admin page.